How to develop the C50M camera driver of rk3588 that running ubuntu-noble system using cross compiler?

Thought for 8 seconds

The user is asking about developing a C50M camera driver for the RK3588 platform running Ubuntu Noble, using a cross compiler. I need to provide a comprehensive guide. I'll search for relevant information.

Found 44 web pages C50M camera driver RK3588 Ubuntu Noble cross compiler

RK3588 camera driver development guide cross compile C50M camera driver Linux

RK3588 Ubuntu Noble cross compile toolchain setup RK3588 camera driver V4L2 example

RK3588 ISP driver development camera

The search results provide various leads. Result 0 might contain general information about Ubuntu-Rockchip. Result 4 mentions C50M and that drivers are in development. Result 0 from the second search seems to be an overview of RK3588 camera driver. Result 2 from the second search might be a MIPI camera driver. Result 0 from the third search is about setting up a cross compiler. Result 2 from the fourth search is camera driver notes. Result 3 from the fifth search is about RK3588 hardware platform camera driver overview. I will open these pages to gather more details.

Read page Ubuntu-Rockchip项目中的Rootfs与内核源码获取指南 ↗

C50M: Camera Module for Raspberry Pi, Nvidia Jetson and Rockchip RK3588 ↗

RK3588硬件平台-摄像头外设驱动overview 以及bringup 调试流程 ↗

Rockchip MIPI Camera Driver ↗ 让NPU跑起来迅为RK3588开发板设置交叉编译器 ↗

Camera driver notes ↗ RK3588硬件平台-摄像头外设驱动overview 以及bringup 调试流程 原创 ↗

The opened pages provide some useful information. The C50M page indicates that drivers are in development. The RK3588 camera driver overview page provides details on driver architecture and bring-up. The camera driver notes page provides device tree configuration details. The cross-compiler setup page provides instructions for setting up a cross compiler. I'll also open some other relevant pages.

Read page 快速上手RK3588：摄像头调试 ↗ RK3588s camera点亮系列（一）驱动框架简介 ↗

RK3588硬件平台-摄像头外设驱动overview 以及bringup 调试流程 ↗

Linux-Kernel Archive: Re: [PATCH v14 00/18] media: rockchip: add a driver for the rockchip c... ↗

Rockchip MIPI Camera Driver ↗

C50M: Camera Module for Raspberry Pi, Nvidia Jetson and Rockchip RK3588 ↗

> The user's question is about developing a camera driver for the C50M module on RK3588 with Ubuntu Noble, using a cross compiler. My response will cover the key aspects: understanding the RK3588 camera system, setting up the cross-compilation environment, obtaining or developing the driver, configuring device tree, building the kernel, testing, and debugging. I will structure the answer to be practical and step-by-step. I'll cite relevant sources. Now I'll formulate the final answer.

Developing a driver for a new camera sensor on the RK3588 platform, especially with Ubuntu Noble, is a systematic process. Here is a comprehensive guide to help you through each step.

🗺️ Step 1: Understand the RK3588 Camera System

Before writing any code, it's crucial to understand the RK3588's camera pipeline. The system uses a Video Input (VIN) framework, which includes:

- **V4L2 (Video4Linux2)**: The core framework for video devices in Linux. Your driver will be a `v4l2_subdev` (sub-device) [41] .
- **Media Controller**: Manages the connections between different hardware blocks like the sensor, the MIPI receiver, and the ISP (Image Signal Processor) [31] .
- **RKCIF (Rockchip Camera Interface)**: The hardware block that captures raw data from the camera via MIPI CSI-2 or DVP interfaces [13] . The RK3588 has six MIPI CSI-2 receivers [39] .
- **ISP (Image Signal Processor)**: The RK3588 features two ISP units that handle image processing tasks like HDR, denoising, and color conversion. These can be used in either "pass-through" (direct) or "read-back" (DDR) modes [11] .

The sensor (e.g., Samsung GN2 in the C50M) is typically connected to the SoC via the MIPI CSI-2 interface [5] .

🔧 Step 2: Set Up Your Cross-Compilation Environment

Since you'll be compiling on an x86 host for the ARM64-based RK3588, a cross-compiler is essential.

1. **Install the Toolchain**: On your Ubuntu host, install the standard ARM64 cross-compiler 🔗 :

```bash
sudo apt update
sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
```

2. **Get the Kernel Source**: Obtain the kernel source matching your target system. The `ubuntu-rockchip` project provides kernels for various Ubuntu versions, including Noble [1] .
3. **Prepare Kernel Headers**: To compile external modules, you'll need the kernel headers for your target kernel. You can either:
   - **Cross-compile the entire kernel** from source to generate the headers.
   - **Install headers on the target device** if available: `sudo apt install linux-headers-$(uname -r)` .

✍️ Step 3: Obtain or Develop the Sensor Driver

This is the core of the task. Your options are:

- **Option A: Check for an Existing Driver**: The C50M uses the **Samsung ISOCELL GN2** sensor [5] . Search the kernel source for a driver (e.g., `gn2` , `s5kgn2`). If one exists, you can adapt it.
- **Option B: Develop a New Driver**: Most sensor drivers follow the V4L2 sub-device model. You can use an existing driver, like for the IMX586 or OV13850, as a reference [41] . The main tasks are implementing:
  - **Probe Function**: Parses device tree settings (clocks, GPIOs, power supplies) and initializes the I2C communication [41] .
  - **I2C Communication**: Functions to read/write sensor registers for configuration.
  - `v4l2_subdev_ops` **Callbacks**: Core functions like `.s_power` , `.s_stream` , and `.g_frame_interval` to control the sensor [41] .
  - `ioctl` **Handler**: Implements sensor-specific controls (e.g., exposure, gain) [41] .

> ⚠️ **Important Note on C50M Driver Status**: As of June 2023, drivers for the C50M on the RK3588 were "in development" . It is **critical to check with the camera vendor (rbts.co) for the latest driver status and any available source code**, as this could save you significant effort.

⚙️ Step 4: Configure the Device Tree

The Device Tree (`.dts`) file is crucial. It tells the kernel how the camera is physically connected.

1. **Define the I2C Device**: Inside your main board's `.dts` file, add a node for the sensor under the appropriate I2C bus. This node should include:
   - `compatible` : A string matching the driver (e.g., `"samsung,gn2"`).
   - `reg` : The sensor's I2C address.
   - `clocks` : Reference to the external clock (e.g., `&cru CLK_MIPI_CAMARA0`).
   - `pinctrl` : Pin control for reset and power-down GPIOs.
   - `rockchip,camera-module-*` : RK-specific properties for the camera engine [41] .
   - `port` : An endpoint node describing the data connection (MIPI, number of lanes, link frequency) [13] .
2. **Link to the VIN**: The sensor's endpoint must be linked to the appropriate `csi2_dcphy` (MIPI receiver) node in the device tree .

🚀 Step 5: Compile and Deploy

You have two primary ways to build the driver.

- **Method 1: In-tree Driver (Recommended)**: Place your driver source file in `drivers/media/i2c/` . Then, cross-compile the entire kernel:

  bash

  ```bash
  export ARCH=arm64
  export CROSS_COMPILE=aarch64-linux-gnu-
  make rockchip_defconfig # Or your board's specific defconfig
  make -j$(nproc) Image modules
  ```

Update the kernel on your RK3588 device with the newly compiled image and modules.

- **Method 2: Out-of-tree Module**: This is useful for development but can be more fragile. You'll need a `Makefile` that uses the kernel headers. A basic template is:

  ```make
  obj-m := c50m.o
  KERNEL_DIR := /path/to/your/kernel/source
  ARCH=arm64
  CROSS_COMPILE=aarch64-linux-gnu-
  all:
      $(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules
  clean:
      $(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean
  ```

  Build the module and copy the generated `.ko` file to your device 🔗 .

🧪 Step 6: Test and Debug

Once booted with the new kernel and driver, verify the setup.

- **Check I2C Communication**: Use `i2cdetect -y <bus>` to see if the sensor responds at its expected address.
- **List Video Devices**: Run `v4l2-ctl --list-devices` to see if your camera appears as a `/dev/video*` node 🔗 .
- **Inspect the Media Pipeline**: Use `media-ctl -p` to check if the sensor is properly linked to the RKCIF and ISP nodes [31] .
- **Capture a Test Image**: Use `v4l2-ctl` to capture a frame.

  ```bash
  # Example: capture 1 frame as a raw file
  v4l2-ctl -d /dev/videoX --set-fmt-video=width=1920,height=1080,pixelformat=NV12
  --stream-mmap --stream-count=1 --stream-to=test_frame.raw
  ```

This process involves low-level kernel work, so patience and attention to detail are key. If you run into specific issues, such as device tree errors or driver probe

failures, feel free to ask. Good luck with your driver development!

44 web pages

I can't understand.

Thought for 2 seconds

> We need to simplify explanation. User says "I can't understand" after a detailed technical guide. So the response should be much simpler, step-by-step, with less jargon. Focus on what they need to do: cross-compile a camera driver for RK3588 running Ubuntu Noble. Maybe they are a beginner. Provide a clear, basic guide: prerequisites, getting source, building a simple driver example, testing. Avoid deep V4L2 internals. Offer to explain any part further.

I apologize for the confusion. Let me break this down into much simpler, actionable steps. I'll assume you have a basic understanding of Linux but are new to driver development.

What you're trying to do

- You have a **RK3588 board** (like Orange Pi 5, Rock 5, etc.) running **Ubuntu Noble**.
- You want to make a **C50M camera** work on it.
- You will use a **cross-compiler** (build on your PC, run on the board).

Step 1 – Get the right kernel source

On your PC (Ubuntu), open a terminal and install the cross-compiler:

```bash
sudo apt install gcc-aarch64-linux-gnu
```

Now download the Rockchip kernel source that matches Ubuntu Noble. The easiest way:

```bash
```

```
git clone https://github.com/rockchip-linux/kernel.git
cd kernel
git checkout develop-5.10   # or whatever version your board uses
```

Step 2 – Prepare the kernel for cross-compilation

Set up environment variables:

```bash
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-
```

Then configure the kernel (use your board's default config, e.g. `rockchip_linux_defconfig`):

```bash
make rockchip_linux_defconfig
make prepare
make modules_prepare
```

This creates the headers needed to build an external driver.

Step 3 – Get or write the C50M driver

The C50M is based on the Samsung GN2 sensor. First check if a driver already exists:

```bash
ls drivers/media/i2c/ | grep -i gn2
```

If not, you'll need to write a new one – but that is complex. A simpler way: **ask the camera vendor** (rbts.co) for a pre-made driver. They often provide a `.c` file and a device tree overlay.

If you have the driver file (e.g. `c50m.c`), copy it into:

```bash
drivers/media/i2c/
```

Then add its name to `drivers/media/i2c/Makefile` and `Kconfig` – but for a quick test you can build it as an **external module**.

Step 4 – Build the driver as an external module

Create a separate folder, e.g. `c50m_driver`, and put your `c50m.c` inside. Also create a `Makefile` with this content:

```make
obj-m := c50m.o
KERNELDIR := /path/to/your/kernel   # e.g. /home/you/kernel
PWD := $(shell pwd)

default:
	$(MAKE) -C $(KERNELDIR) M=$(PWD) modules

clean:
	$(MAKE) -C $(KERNELDIR) M=$(PWD) clean
```

Now build it:

```bash
make
```

If successful, you'll get a file `c50m.ko`. Copy it to your RK3588 board (using scp or USB drive).

Step 5 – Install and test on the board

On the RK3588 board (running Ubuntu Noble):

```bash
```

```
sudo insmod c50m.ko
```

Check if the driver loaded:

```bash
dmesg | tail
lsmod | grep c50m
```

If you see errors, the driver may need a **device tree** configuration. That is another step: you have to modify the board's device tree to tell the kernel where the camera is connected (I2C bus, GPIO reset, etc.).

I know this is still technical – ask me about one piece

Which part is hardest for you?

- "I don't have the driver source code" → I can guide you on how to write a minimal skeleton.
- "I don't understand the device tree" → I can explain with a simple example.
- "The driver builds but doesn't work" → I can help you debug with `dmesg` and `i2cdetect`.

Just tell me where you are stuck, and I'll give you a short, focused answer.

I don't have the C50M driver source code.

Thought for 4 seconds

> The user needs to develop a C50M camera driver for RK3588 but lacks the source code. I need to provide guidance on how to proceed without the source code. This involves searching for existing drivers, creating a new driver from scratch, using similar drivers as templates, and contacting the vendor. I will follow the search plan as outlined.

Found 55 web pages C50M camera sensor driver Linux

C50M sensor datasheet specifications RK3588 camera driver development V4L2 subdev
Samsung GN2 driver Linux kernel Rockchip ISP camera sensor driver template
how to write V4L2 sensor driver from scratch C50M Samsung GN2 sensor RK3588

> The search results show that the C50M camera module uses the Samsung GN2 sensor. There is no specific driver source code available. The results include some documentation and guides. I need to provide guidance on how to proceed without the source code. I will structure the response by covering the following: confirming the sensor and checking for a driver, contacting the vendor, building a driver from scratch, and other options like using a similar driver as a template. I will cite relevant sources.

It's completely understandable to feel stuck without the source code. Fortunately, that's the first challenge we can tackle head-on. Let's map out a plan to get you the driver code or to build a viable alternative.

Here's a strategic plan to move forward:

🎯 Step 1: Start by Confirming Your Hardware

First, let's make absolutely sure we're looking for the right thing. The "C50M" is a camera module, and the actual image sensor on it is the **Samsung ISOCELL GN2** 🔗 . This detail is your key to unlocking the next steps.

🗺️ Step 2: Your Action Plan

Here's a clear, step-by-step plan you can follow, starting with the easiest option:

1. Do an Exhaustive Search (The Quickest Win)
Before writing a single line of code, perform a very specific search for the driver.

- **What to search for**: Use the actual sensor's name: `"Samsung S5KGN2 Linux driver"` or `"S5KGN2 kernel module"` . The "C50M" is the product name, but the code will be named after the sensor.
- **Where to look**:
  - In your downloaded kernel source: `find ./ -name "*gn2*"` .
  - On GitHub and GitLab.
  - On forums dedicated to RK3588 boards like the Orange Pi 5 or Rock 5 Model B 🔗 .

2. Contact the Vendor (The Most Direct Route)
The vendor, **RBTS.co**, has publicly stated that the C50M module is designed for

the RK3588 . This is a very strong indicator they have, or are developing, the driver. Reach out to them directly and ask for their driver source code or a pre-built kernel module.

3. If All Else Fails: Build the Driver from Scratch
If you can't find an existing driver and the vendor doesn't respond, you'll need to write it yourself. This is a significant but achievable project. I've found a **very detailed, step-by-step tutorial** specifically on developing a V4L2 sensor driver from scratch .

- **The Framework**: Your driver will be a `v4l2_subdev` (sub-device), which is the standard Linux way of controlling a camera sensor .
- **The Core Tasks**: The driver's main job is to talk to the sensor over I2C to set things like resolution and frame rate, and to manage its power states .

To get started with this approach, we'd need to follow a process:

- **Find the Datasheet**: You'll need the programming manual for the S5KGN2 sensor. This document lists all the register addresses you need to configure.
- **Use a Template**: Look at the source code of an existing driver (like `imx319.c`) as a reference for the correct structure and API calls .

⚙️ An Easier Alternative: Find a Similar Driver

An intermediate step you can try is to find a driver for a similar sensor from the same family, like the Samsung S5KJN1, and adapt it. While the low-level register settings will be different, the overall structure of the driver might be very similar, saving you a lot of work.

Since building a driver from scratch is a major task, what's your comfort level with tackling this next step on your own? If you're ready to start coding, I can guide you through writing a basic driver skeleton, or we can look for an existing similar driver to adapt. Let me know how you'd like to proceed!

55 web pages

Give me completed C50M camera driver code.

Thought for 14 seconds

The user wants a complete driver code for the C50M camera on RK3588 with Ubuntu Noble. I need to search for the driver. I will search for "C50M camera driver RK3588" and also "Samsung GN2 driver" as the C50M likely uses that sensor. I'll also search for "C50M driver source code" and "RK3588 camera driver development".

Found 24 web pages C50M camera driver RK3588 Samsung GN2 driver Linux kernel "C50M" driver source code RK3588 camera driver development

The search results for "C50M camera driver RK3588" show a crowdfunding page and a forum discussion, but no driver source code. The results for "Samsung GN2 driver Linux kernel" are not relevant. The search for "C50M driver source code" returned no results. The search for "RK3588 camera driver development" yielded some repositories, but not specifically for the C50M. I need to search more specifically for "RBTS.co C50M driver" and "C50M driver download".

Found 9 web pages RBTS.co C50M driver download "C50M" "driver" download

The search results for "C50M driver download" are not relevant. It seems the driver is not publicly available. I need to provide a guide on how to develop the driver, including a skeleton driver. I'll also search for "S5KGN2 driver" and "Samsung ISOCELL GN2 driver".

Found 5 web pages "S5KGN2" driver "ISOCELL GN2" driver linux

No driver found. I will search for "Samsung 50MP sensor driver Linux" and "MIPI CSI-2 sensor driver template".

Found 19 web pages Samsung 50MP sensor driver Linux MIPI CSI-2 sensor driver template Linux v4l2

The search results for "Samsung 50MP sensor driver Linux" show a driver for the S5KJN1, which might be similar to the GN2. The search for "MIPI CSI-2 sensor driver template" didn't yield a template. I'll open result 0 from the Samsung 50MP search.

Read page media: i2c: add Samsung S5KJN1 image sensor device driver ↗

This page shows a driver for the S5KJN1, which might be similar to the GN2. I'll look for the source code. I'll search for "s5kjn1.c" and "s5kgn2.c".

Found 19 web pages s5kjn1.c filetype:c "s5kgn2" "media" "i2c"

> No driver source found. I'll search for "S5KGN2 driver source code" and "S5KGN2 kernel driver".

Found 10 web pages "S5KGN2" "kernel" driver "s5kgn2" "v4l2"

> It appears that no public driver source code for the C50M or S5KGN2 sensor is available. The user likely wants a complete driver, but it doesn't exist publicly. I need to set expectations: the driver is not publicly available, and I cannot provide the source code. I should explain why it's not available and offer alternatives. The response should include: a detailed framework to adapt from a similar driver, where to find official support, and the steps to adapt a similar driver. I'll cite the sources found. I'll now craft the response.

I understand you're looking for the complete driver code. However, I must set a realistic expectation: **The C50M driver for RK3588 is not publicly available as open source**, as RBTS.co (the manufacturer) states the drivers are still "in development" . No public driver code exists for either the C50M or its underlying Samsung S5KGN2 sensor .

Since I cannot provide a ready-to-use driver, I will instead provide you with a comprehensive blueprint to develop one yourself, focusing on the critical adaptation of a very similar driver and guidance on obtaining official support.

🗺️ Plan of Action for Obtaining or Building the Driver

Here is the strategic path forward:

1. 🎯 Prioritize Getting the Driver from RBTS.co (The Most Effective Route)

Since RBTS.co is developing these drivers, they are your best source. You should contact them directly. Prepare a technical inquiry for their support team at **rbts.co**, clearly stating:

- Your target platform (RK3588) and operating system (Ubuntu Noble).
- That you need the driver source code or a pre-built kernel module.
- Your development timeline and any purchasing details (if you already bought the module).

2. ✍️ Develop the Driver by Adapting a Similar Sensor (If You Must Build It Yourself)

This is your fallback plan. A driver for the **Samsung S5KJN1**—a 50MP sensor with very similar specifications—was recently added to the Linux kernel [39] . You can use this as your blueprint.

Here is the step-by-step adaptation framework:

Step	Action	Key Points
1. Locate the Blueprint	Get the `s5kjn1.c` driver source from the latest Linux kernel.	Search for it in the `drivers/media/i2c/` directory of the kernel source.
2. Create a Copy	Copy `s5kjn1.c` to a new file named `s5kgn2.c` .	This is your working base.
3. Update the Basics	Modify the `compatible` string and driver description.	Change "s5kjn1" to "s5kgn2" in the `struct i2c_device_id` and `struct of_device_id` tables.
4. Find the Register Map	Source the S5KGN2's register map.	This is the most difficult part. Check with RBTS.co, search for leaked Android kernel sources, or reverse-engineer from the S5KJN1 driver if the registers are similar.
5. Update Resolution Modes	Modify the `struct s5kjn1_mode` array for S5KGN2's max resolution (8160x6144).	Update the register lists for each mode based on your register map 🔗 .

Step	Action	Key Points
6. Re-calculate Timing	Adjust PLL and timing calculations.	Update functions that calculate link frequency and pixel rate for the S5KGN2's specs.
7. Update the Build System	Modify `Kconfig` and `Makefile`.	Add your `SENSOR_S5KGN2` config option and `s5kgn2.o` to the build.
8. Adapt the Device Tree	Create a `.dtsi` fragment for your new sensor.	Include properties for I2C address, reset GPIOs, power supplies, and the MIPI data-lanes.

3. ⚙️ Using the Driver on RK3588 with Ubuntu Noble

Once you have the driver code, here is the development and testing pipeline:

1. **Develop as an Out-of-Tree Module**: Use a `Makefile` to compile the driver against your target's kernel headers for faster iteration.
2. **Deploy and Test**: Transfer the `.ko` file to your RK3588 board, load it with `insmod`, and verify the driver attaches to the I2C bus.
3. **Validate the Video Pipeline**: Use `media-ctl` and `v4l2-ctl` to verify the sensor is properly linked to the RKCIF and ISP blocks and to capture test frames.

⚠️ Important Considerations for the Adaptation

- **Register Map**: This is the single biggest challenge. The S5KGN2 is a more advanced sensor than the S5KJN1 🔗, so the register maps will be different. Without this, you cannot complete the adaptation.
- **RK3588 Integration**: The RK3588 is a complex SoC with specific requirements for MIPI CSI-2 and ISP configuration. Ensure your driver correctly implements the `v4l2_subdev` and media controller interfaces to integrate with Rockchip's VIN (Video Input) framework 🔗.

I know this is a challenging task, but you now have a clear, actionable plan and a real blueprint to start from. What is your next step? Do you want to start with contacting RBTS.co, or shall we begin adapting the `s5kjn1.c` driver and I'll guide you through the first few steps?

82 web pages

Show me the completed driver code.

Thought for 3 seconds

> The user is asking for the completed driver code for the C50M camera on the RK3588 platform. They are likely a developer who wants to integrate this camera. I need to provide a complete driver code. I will search for the driver code.

Found 27 web pages C50M camera driver rk3588 C50M camera driver source code rk3588 camera driver C50M github "C50M" "rk3588" driver C50M camera module rk3588 driver download

> The search results show that the driver is still in development and not publicly available. I need to provide a complete driver code, but it doesn't exist. I should explain this clearly and offer alternatives. I'll structure my response to first state the reality, then offer to help find a similar driver or provide a template.

I understand your need for the completed code. As of my knowledge, a complete, ready-to-use driver for the C50M module on the RK3588 platform has not been publicly released yet.

The manufacturer, **RBTS.co**, has stated that drivers for the C50M are still **"in development"** for all platforms, including the RK3588 . This means that a finished driver is not available for me to provide you. The only way to get the official driver is to contact RBTS.co directly or wait for their official release announcement .

While I can't give you the completed driver, I can offer a few practical next steps:

- **Check Similar Projects**: I can search for existing drivers for cameras that are **very similar** to the C50M, such as the **Samsung ISOCELL GN2**, or for **comparable 50MP sensors**. Using a similar driver as a template is often the best starting point.

- **Review Official Resources**: I can help you check the official RBTS.co website and the Radxa forum for any new updates, code snippets, or community-driven projects that might have made progress on this.
- **Provide a Driver Template**: If you're ready to start coding, I can provide you with a **skeleton driver** that is designed for the RK3588 and uses the standard V4L2 framework. This would give you a working structure to start filling in.

Which of these paths would you like to explore first?

27 web pages

Where is the s5kjn1.c file?

Thought for 4 seconds

> The user is asking where to find the s5kjn1.c file. I need to search for this file in the Linux kernel source. I will perform a search to locate it.

Found 8 web pages s5kjn1.c file location Linux kernel source

> The search results show some relevant links. I need to open some of them to find the exact location of the s5kjn1.c file.

Read page [PATCH v4 0/2] media: i2c: add Samsung S5KJN1 image sensor device driver media: i2c: add Samsung S5KJN1 image sensor device driver linux.git - Linus' kernel tree

> The search results show that the s5kjn1.c file is part of the Linux kernel source. I need to provide the user with information on where to find it. I will mention that it's located in drivers/media/i2c/ of the Linux kernel source tree. I can also provide a direct link to the file in the kernel repository.

The `s5kjn1.c` driver is currently being integrated into the mainline Linux kernel. You can find it in the `drivers/media/i2c/` directory of the kernel source tree. [6] While it may not be in your current kernel source, you have several options to obtain it.

💻 Option 1: Use the Mainline Linux Kernel Git Repository

The most straightforward method is to clone the mainline kernel repository and check out the specific commit where the driver was added. This gives you the

complete, official source code.

- **Clone the repository**:

```bash
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux
```

- **Check out the commit**: The driver was added in commit `e38fd0933c75` [6]. Switch to that state:

```bash
git checkout e38fd0933c75
```

- **Locate the driver**: You will find the source code and its corresponding build file here:
  - Source file: `drivers/media/i2c/s5kjn1.c`
  - Build file: `drivers/media/i2c/Makefile`

💻 Option 2: Locate the File in an Existing Kernel Source

If you already have a Linux kernel source tree on your system, you can search for the file using the `find` command:

```bash
find /path/to/your/kernel/source -type f -name "s5kjn1.c"
```

If the driver is present, this command will print its full path.

💻 Option 3: View the Source Code Online

You can also view the file online using the Linux kernel's cgit web interface:

- **Link to the file**: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/me

dia/i2c/s5kjn1.c?h=v6.14

- **Raw file**: For a plain-text view suitable for copying, you can use the raw link: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/plain/drivers/media/i2c/s5kjn1.c?h=v6.14

I hope these options help you access the driver source code. Let me know if you need any further guidance.

8 web pages